What Transfers Through Mathematical Distillation?
A Matched Study of Task Form and Training Method

Aryama V Murthy 2024101043
Mukund Hebbar 2024101037
Yashas Nataraj Kotre 2024101028
Anivarth Varma Penumetcha 2024111006

Abstract

Mathematical distillation is usually evaluated by whether a student model reaches the correct final answer. A useful model must also be able to plan a solution and check whether a proposed solution is correct. We study whether training on one of these three task forms improves the other two. A teacher converts each source problem into matched SOLVE, PLAN, and JUDGE questions that use the same mathematical idea. We train Qwen3-1.7B on one form at a time and test every trained model on all three forms. We compare distillation from fixed teacher responses with reverse-KL on-policy distillation, where the teacher evaluates responses generated by the student during training. This produces a table showing which improvements remain limited to the trained form and which extend to other forms. We then test whether this table can guide the choice and ordering of data in mixed-task training.

1 Introduction and Motivation

Models are often trained and tested on one kind of mathematical question. This does not show whether an improvement can be used in another way. A student may learn to produce correct answers without becoming better at planning a solution or checking another solution. This matters when a model is expected to perform all three tasks after being trained with a limited amount of data.

Existing benchmarks show that these abilities should be measured separately. MathBench tests knowledge of mathematical theory and the application of that knowledge as separate parts of the benchmark. It reports a clear gap between them (Liu et al., 2024). ProcessBench asks models to decide whether a proposed solution is correct and, when it is wrong, identify the first incorrect step. Its results show that solving a problem and checking a reasoning process are not the same ability (Zheng et al., 2025). SymPyBench also finds that models can answer one form of a problem correctly and another form incorrectly, even when the underlying problem is matched (Imani et al., 2026).

Our goal is to measure how training moves between these abilities under a controlled setup. We train a fresh copy of the same student on only one task form and then test it on all three forms. The source concepts, topic distribution, number of training problems, student checkpoint, and evaluation problems are held fixed. We ask:

1. If the student is trained on one form, which other forms improve, remain unchanged, or get worse?
2. Do fixed teacher responses and reverse-KL on-policy distillation produce different patterns of improvement across forms?
3. Can the measured results help choose the proportions and order of data in mixed-task training?

2 Background and Related Work

Teacher-generated reasoning can provide more useful supervision than answer labels alone. Distilling Step-by-Step shows that training on teacher explanations can improve smaller models with less data (Hsieh et al., 2023). On-policy distillation instead lets the student generate its own response and asks the teacher to score the next-token choices along that response. This exposes the teacher to mistakes that the current student actually makes (Agarwal et al., 2024). We compare this method with ordinary off-policy distillation from teacher responses created before training.

Several studies examine transfer between particular mathematical tasks. PLAN-TUNING shows that training on plans can improve solving on unseen benchmarks (Parmar et al., 2025). VeriThinker reports that verification-only training can preserve or slightly improve answer accuracy while shortening reasoning (Chen et al., 2025). Learning to Self-Verify directly studies transfer between generation

and verification. It finds that generation training does not necessarily improve verification, while verification training can improve generation (Chen et al., 2026). Krishna et al. measure how training on one dataset changes performance on other NLP tasks, but those tasks use different datasets and input distributions (Krishna et al., 2025).

Other work uses relationships between tasks to decide how training data should be allocated. Skill-It estimates which skills support other skills and changes their sampling proportions during training (Chen et al., 2023). ReAD studies how distillation on one capability changes other capabilities and adaptively divides a fixed token budget (Cheng et al., 2026). SMART instead chooses a fixed mixture of instruction-tuning tasks under a limited data budget (Renduchintala et al., 2024). These studies motivate us to test a task order chosen from development results rather than assume that it will help.

The closest studies therefore establish that transfer can be unequal in opposite directions. They do not compare SOLVE, PLAN, and JUDGE together using matched mathematical concepts, the same student, and equal numbers of source problems. They also do not compare the resulting pattern under fixed-response and on-policy distillation. Our study fills this narrower gap.

3 Matched Task Forms

We create three versions of every retained source problem. Each version changes what the model must do while keeping the main mathematical concept and intended solution method fixed. Variants from the same source problem always remain in the same data split.

Before generation, we store the source problem, its answer, a valid derivation, the main method, and any necessary assumptions. The teacher uses this record to create the three forms in Table 1. Numerical values may change between forms so that one answer cannot be copied into another form. We retain a set only when the intended method still applies and the approximate difficulty remains similar.

For JUDGE, the teacher creates both correct and incorrect proposed solutions. We aim for an equal number of each. Incorrect solutions include a marked first error, and we vary the position and type of error so that the model cannot rely on one repeated pattern. Automatic checks verify answers and any algebraic or numerical claims that can be executed. A human audit checks a random sample for mathematical correctness, preservation of the intended method, and comparable difficulty. Invalid sets are corrected or removed.

Form	Required output
SOLVE	Work through the problem and give the correct final answer.
PLAN	State a valid method and the first useful steps, without completing the calculation.
JUDGE	Decide whether a proposed solution is correct. If it is wrong, identify the first step where it becomes wrong and explain why.

Table 1: The three task forms created from each source problem.

4 Data and Models

4.1 Training and development data

Training uses text-only problems from DeepMath-103K, which provides difficult problems with verifiable answers and solution traces (He et al., 2026). We select problems from algebra, number theory, combinatorics, discrete mathematics, and geometry that does not require an image. We use 4,096 source problems and create all three forms for each one, giving 4,096 training examples per form. A separate held-out part of DeepMath is used for prompt development, checkpoint selection, mixture selection, and curriculum decisions. Topic and difficulty distributions are kept as similar as the valid problems allow.

4.2 Final evaluation data

The final test set contains 1,524 source problems. We reserve 1,024 held-out DeepMath problems and add 500 problems drawn in total from the text-only parts of OlympiadBench and MathBench (He et al., 2026, 2024; Liu et al., 2024). Each source is converted into all three forms, giving 4,572 test instances. Problems that cannot be converted without changing the main concept or requiring an image are excluded using rules fixed before evaluation. Variants of a test problem are never used in training or development. The final test set is used only after all prompts, checkpoints, mixtures, and training orders have been fixed.

4.3 Student, teacher, and judge

The student is Qwen3-1.7B and the teacher is Qwen3-8B (Yang et al., 2025). Both use their

instruction-following, non-thinking mode. In practice, we apply the chat template with thinking disabled and keep this setting fixed for data generation, training, and evaluation. The two models share a tokenizer, which allows the teacher and student next-token distributions to be compared directly. A pilot must confirm that the 8B teacher outperforms the unchanged 1.7B student on all three development tasks.

A separate strong model, such as DeepSeek-V4 Flash, grades open-ended outputs. It is not used to generate training targets or provide distillation probabilities. We fix the judge model, prompt, decoding settings, and rubric before the final evaluation.

5 Experimental Design

5.1 Training one form at a time

Let M_0 be the unchanged student. For each form T_i, we train a fresh copy of the student on only that form. We then test the trained model on SOLVE, PLAN, and JUDGE. We repeat this procedure for two training methods and two random seeds.

Off-policy distillation. The 8B teacher first produces one target response for every training prompt. The student is then trained by next-token prediction on these fixed responses. The targets do not change as the student changes.

Reverse-KL on-policy distillation. The student generates responses during training. The teacher then provides next-token probabilities along the student's generated sequence (Agarwal et al., 2024). For prompt x and a student-generated history h, we minimize

$$\mathcal{L}_{\mathrm{R}} = \mathbb{E}_{x,h\sim p_S}\left[D_{\mathrm{KL}}\left(p_S(\cdot \mid x,h) \,\|\, p_T(\cdot \mid x,h)\right)\right]. \tag{1}$$

This loss reduces probability on tokens that the student prefers but the teacher considers unlikely. Unlike fixed-response training, it provides feedback on the sequences produced by the current student.

Every run uses the same 4,096 source problems, number of training examples, optimization settings, and maximum generation length of 4,096 tokens. Generation may stop earlier when an end token is produced. We report the actual number of generated and training tokens because the three forms naturally have different response lengths. We save checkpoints during training to measure whether changes appear early or only near the end.

5.2 Measuring change across forms

Let $S_j(M)$ be model M's score on test form T_j. We subtract the unchanged student's score from the trained student's score:

$$G_{i\to j}^{(m)} = S_j\left(M_i^{(m)}\right) - S_j(M_0). \tag{2}$$

When $i = j$, this is the improvement on the form used for training. When $i \neq j$, it shows whether training on one form changes performance on another. For each method, we report the nine values in a 3×3 table. We include negative values and compare opposite directions, such as training on SOLVE and testing on JUDGE versus training on JUDGE and testing on SOLVE.

5.3 Training checkpoints and GradAlign

GradAlign was introduced for GRPO. It selects training problems whose gradients align with those of a trusted development set (Yang et al., 2026). A related OPD diagnostic compares a distillation gradient with an estimated ideal success-improving gradient, but it does not use development-set alignment to select problems (Armandpour et al., 2026). We explore whether the GradAlign principle extends to on-policy distillation by computing alignment with OPD gradients instead of policy gradients. We test whether early OPD gradient alignment predicts later cross-task transfer and can provide a useful signal for curriculum construction. This analysis uses only the training and development sets. The final test set is not used to choose examples or make predictions.

6 Evaluation

SOLVE is graded with deterministic checks. We normalize equivalent answer formats and use exact match or direct substitution where appropriate. PLAN is graded on whether the proposed method is valid and whether the stated steps are sufficient to continue toward the reference answer. JUDGE is graded on the correctness decision and, for an incorrect solution, whether the model identifies the first wrong step and explains the error correctly. The fixed external judge applies separate rubrics for PLAN and JUDGE. We manually grade a random sample of these outputs and report agreement with the external judge.

The score for each form is reported separately. We also report the average across the three forms as a secondary summary. Results include both seeds. Confidence intervals are obtained by resampling source problem IDs, which keeps the three related forms together. We correct for the six comparisons per method where the training form and test form differ. Difficulty groups are reported separately when each group contains enough test problems.

7 Transfer-Guided Mixed Training

The single-form experiments first show how training on each form changes the other two. We then test whether these development-set results can guide mixed training.

We assume a deployment distribution over SOLVE, PLAN, and JUDGE. This distribution represents how often each ability is expected to be used. We use the development-set results to identify a promising training mixture with the highest predicted average gain under this distribution, while requiring that SOLVE performance does not decrease. We train this mixture directly and compare it with SOLVE-only training and an equal mixture of all three forms. The transfer results are used to choose a candidate mixture, not as an exact prediction of its final performance.

We also test whether unequal transfer in opposite directions can guide training order. If training on one form helps another more than the reverse direction, we give the first form more weight earlier and the second form more weight later. We compare this soft curriculum with a randomly shuffled mixture containing the same examples and the same overall task proportions. Only the order changes. The deployment distribution, selected mixture, and training order are chosen on the development set and fixed before final evaluation.

8 Expected Contributions

The project makes three contributions. First, it provides a matched dataset for testing how the same mathematical concept is used for solving, planning, and judging. Second, it measures how training on each form changes performance on all three forms under fixed-response and reverse-KL on-policy distillation. Third, it tests whether the measured changes can help choose the proportions and order of examples in mixed-task training.

The main contribution is the controlled comparison. The results will show whether an improvement extends to other uses of the same mathematical concept or remains limited to the response format used during training.

9 Limitations

The three forms cannot be separated perfectly. A detailed plan can reveal much of a solution, and judging a solution may require solving part of the problem. Numerical changes can also alter difficulty even when the intended method stays the same. Teacher-generated questions and proposed solutions may contain errors after automatic checks, so human auditing remains necessary. Grading PLAN and JUDGE depends partly on an external model. Results from one student family and selected mathematics datasets may not apply to every model or domain. Finally, transfer measured after single-form training may not combine predictably when tasks are mixed. We therefore evaluate every selected mixture directly.

10 Tentative Timeline

Period	Work
Early–mid September	Finalize prompts, the evaluation setup, and the training pipeline.
Late September	Run one seed of one distillation method for SOLVE, PLAN, and JUDGE. Report preliminary transfer results in the mid submission.
Early–mid October	Complete the remaining distillation runs and second seeds, then construct the final transfer matrices.
Mid–late October	Run the mixture and curriculum experiments, analyze results, and prepare the final report.

Table 2: Tentative project timeline.

References

Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. 2024. On-policy distillation of language models: Learning from self-generated mistakes. In *International Conference on Learning Representations*.

Mohammadreza Armandpour, Fatih Ilhan, David Harrison, Ajay Jaiswal, Duc N. M. Hoang, Fartash Faghri, Yizhe Zhang, Minsik Cho, and Mehrdad Farajtabar. 2026. Unmasking on-policy distillation: Where it helps, where it hurts, and why. *arXiv preprint arXiv:2605.10889*.

Mayee F. Chen, Nicholas Roberts, Kush Bhatia, Jue Wang, Ce Zhang, Frederic Sala, and Christopher Ré.

2023. Skill-it! a data-driven skills framework for understanding and training language models. In *Advances in Neural Information Processing Systems*, volume 36, pages 36000–36040.

Yuxin Chen, Yu Wang, Yi Zhang, Ziang Ye, Zhengzhou Cai, Yaorui Shi, Qi Gu, Hui Su, Xunliang Cai, Xiang Wang, An Zhang, and Tat-Seng Chua. 2026. Learning to self-verify makes language models better reasoners. *arXiv preprint arXiv:2602.07594*.

Zigeng Chen, Xinyin Ma, Gongfan Fang, Ruonan Yu, and Xinchao Wang. 2025. VeriThinker: Learning to verify makes reasoning model efficient. In *Advances in Neural Information Processing Systems*.

Xueqi Cheng, Xugui Zhou, Tyler Derr, and Yushun Dong. 2026. ReAD: Reinforcement-guided capability distillation for large language models. *arXiv preprint arXiv:2605.11290*.

Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, Jie Liu, Lei Qi, Zhiyuan Liu, and Maosong Sun. 2024. OlympiadBench: A challenging benchmark for promoting AGI with olympiad-level bilingual multimodal scientific problems. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 3828–3850.

Zhiwei He, Tian Liang, Jiahao Xu, Qiuzhi Liu, Xingyu Chen, Yue Wang, Linfeng Song, Dian Yu, Zhenwen Liang, Wenxuan Wang, Zhuosheng Zhang, Rui Wang, Zhaopeng Tu, Haitao Mi, and Dong Yu. 2026. DeepMath-103K: A large-scale, challenging, decontaminated, and verifiable mathematical dataset for advancing reasoning. In *International Conference on Learning Representations*.

Cheng-Yu Hsieh, Chun-Liang Li, Chih-Kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alexander Ratner, Ranjay Krishna, Chen-Yu Lee, and Tomas Pfister. 2023. Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 8003–8017.

Shima Imani, Seungwhan Moon, Adel Ahmadyan, Lu Zhang, Ahmed Kirmani, and Babak Damavandi. 2026. SymPyBench: A dynamic benchmark for scientific reasoning with executable Python code. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics: Industry Track*.

Shambhavi Krishna, Atharva Naik, Chaitali Agarwal, Sudharshan Govindan, Haw-Shiuan Chang, and Taesung Lee. 2025. Latent traits and cross-task transfer: Deconstructing dataset interactions in LLM fine-tuning. In *Proceedings of the 14th Joint Conference on Lexical and Computational Semantics*, pages 225–241.

Hongwei Liu, Zilong Zheng, Yuxuan Qiao, Haodong Duan, Zhiwei Fei, Fengzhe Zhou, Wenwei Zhang, Songyang Zhang, Dahua Lin, and Kai Chen. 2024. MathBench: Evaluating the theory and application proficiency of LLMs with a hierarchical mathematics benchmark. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 6884–6915.

Mihir Parmar, Palash Goyal, Xin Liu, Yiwen Song, Mingyang Ling, Chitta Baral, Hamid Palangi, and Tomas Pfister. 2025. PLAN-TUNING: Post-training language models to learn step-by-step planning for complex problem solving. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 21419–21433.

H S V N S Kowndinya Renduchintala, Sumit Bhatia, and Ganesh Ramakrishnan. 2024. SMART: Submodular data mixture strategy for instruction tuning. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 12916–12934.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Ningyuan Yang, Weihua Du, Weiwei Sun, Sean Welleck, and Yiming Yang. 2026. GradAlign: Gradient-aligned data selection for LLM reinforcement learning. *arXiv preprint arXiv:2602.21492*. Accepted at COLM 2026.

Chujie Zheng, Zhenru Zhang, Beichen Zhang, Runji Lin, Keming Lu, Bowen Yu, Dayiheng Liu, Jingren Zhou, and Junyang Lin. 2025. ProcessBench: Identifying process errors in mathematical reasoning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, pages 1009–1024.